

Presentation 1: Understanding Problem & Fall 2025 Work

STAT 390 | Group 2 | Winter 2026

Adithi Adusumilli, Jason Chen, Roma Hashemi, Charlene Lu & Paige Usry

Part 1: Programmers

Understanding *main.py*

- This file coordinates the data pre-processing, training, evaluation, and the analysis through the components defined in other files
- Essentially the central point for running the full MIL
- Invoked when submitting the job by the sbatch
 - `main..py` doesn't implement the algorithm itself but it organizes and executes it
- Whole workflow from the raw data to the results files
 - Sets up the experiment environment by setting the seed and defining the directories
 - Prepares the data into clean train/val/test datasets
 - Initialize and trains the model
 - Evaluates the performance and analyzes the outputs

4 core functions: `parse_args()`, `prepare_data()`, `create_data_loaders()`, `main()`

Understanding *main.py*

```
def parse_args():  
    ...  
    return parser.parse_args()
```

Main Idea:

This function essentially sets the experiment settings and specifies which parameters can be controlled at runtime, assigning default values from *config.py*



config.py is a configuration file for the MIL training that defines default paths, model architecture, training behavior, data splits, and preprocessing

Inputs: none (from command line)

Outputs: the parser arguments for the experiment

Types of arguments:

- Data (labels, patches)
- Training (epochs, learning rate, batch size, number of workers)
- Model (embedding dimension, per slice cap, maximum slices per stain)
- Other (seed, resume, evaluate only, analyze attention, attention top n, load splits)

Understanding *main.py*

```
def prepare_data(args):  
    ...  
    return (train_case_dict, train_label_map),  
           (val_case_dict, val_label_map),  
           (test_case_dict, test_label_map)
```

Main Idea:

Prepares raw data (the patch files and ground truth labels) into train/val/test dataset splits to be used for the MIL

- Loads the case-level labels
- Collects all the patch files and groups them into slices and then into their cases
- Temporarily assign case labels to slices for splitting
- Case-level stratified splitting (high grade, benign)
- Leakage and integrity checks

Inputs: args (data paths, random seed, optional saved splits)

Outputs: (train, val, test) case dictionaries and label maps

***data_utils.py** - processing utilities for grouping the patches into slices and cases, assigning labels, stratified splitting, and preventing leakage*

***utils.py** - utilities for reproducibility, validation, logging, and saving/loading the dataset splits*

***trainer.py** - used for dataset diagnostics like patch count summaries by class*

Understanding *main.py*

```
def create_data_loaders(train_data, val_data, test_data, args):  
    ...  
    return train_loader, val_loader, test_loader
```

Inputs: training, validation, and test data;
arguments

Outputs: training, validation, and test data
loaders

Main Idea:

This function takes the prepared data, turns it into PyTorch datasets, and wraps them in loaders that the training loop can iterate over efficiently

- Dataset: what data exists and how to access it
- Data Loader: how to batch, shuffle, feed to model

dataset.py - defines the MIL dataset structure, the patch sampling strategy, data transforms, and collate function

PyTorch (*torch.utils.data*) - provides *DataLoader* infrastructure for efficient iteration

Understanding *main.py*

```
def main():  
    ...  
    print(f"\nResults saved to: {run_dir}")  
    print("Training completed successfully!")
```

Inputs: command line arguments

Outputs: trained model checkpoints,
predictions, eval. metrics, attention visuals

models.py - implements the hierarchical attention
MIL model architecture

trainer.py - implements the training / evaluation logic

attention_analysis.py - performs post-hoc analysis
and visualization for interpreting the model and the
assigned attention weights

Main Idea:

This function calls the previously defined functions to run the full experiment, by executing the pipeline from configuration to training, evaluation, and analysis

Workflow:

- Reads configuration (from `parse_args`), sets up reproducibility and environment
- Prepares data and splits (from `prepare_data`)
- Builds dataset and loaders (from `create_data_loaders`)
- Creates the model and trainer
- Handles training, evaluation, checkpointing, and optional analysis
- Saves results and logs

Functions Legend

attention_analysis.py	data_utils.py	utils.py
• analyze_attention_weights() • analyze_case_attention() • visualize_patch_attention() • visualize_case_effective_patches() • save_attention_summary() • plot_attention_distribution() • compute_effective_patch_attention() • plot_effective_patch_attention_distribution_per_case() • analyze_top_effective_patches_per_case() • plot_slice_attention_distribution_per_case_and_stain()	• load_labels() • get_all_patch_files() • group_patches_by_slice() • build_slice_to_class_map() • split_by_case_stratified() • build_case_dict() • extract_stain_from_filename() • get_case_ids() • get_all_paths() • check_disjoint_sets() • report_no_leak() • summarize_case_dict()	• set_seed() • get_device() • print_data_summary() • create_run_directory() • save_data_splits() • load_data_splits() • print_model_summary() • analyze_attention_weights() • check_data_integrity() • format_time() • get_memory_usage()

Stratified Splitting in Code

How cases, slices, and patches are organized before the split



```
# Load labels
labels = load_labels(args.labels_csv)
print(f"Loaded {len(labels)} labels")
```

1. Load ground truth labels at the CASE level (only true labels)

```
# Get patch files
all_files = get_all_patch_files(args.patches_dir)
print(f"Found {len(all_files)} patch files")
```

2. Load all patch files, count patches

```
# Group patches by slice
patches = group_patches_by_slice(all_files, args.patches_dir)
print(f"Grouped into {len(patches)} slices")
```

3. Group patches by the slice they came from, count slices

```
# Build slice to class mapping
slice_to_class = build_slice_to_class_map(patches, labels)
print(f"Mapped {len(slice_to_class)} slices to classes")
```

4. Temporary case labels for the slices (for splitting only)

- Each slice inherits its case label
- Used only for purposes of stratified splitting
- Patches themselves are never labeled

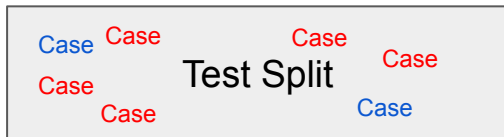
```
# Group slices by class for stratified splitting
slices_by_class = defaultdict(list)
for key, label in slice_to_class.items():
    slices_by_class[label].append(key)
```

```
=====
PREPARING DATA
=====
Loaded 105 labels
Found 83595 patch files
All 3808 non-standard file names were handled.
Grouped into 378 slices
Mapped 366 slices to classes
Class distribution: {0: 128, 1: 238}
```

Stratified Splitting in Code

Ground truth = high grade case
Ground truth = benign case

How the final model structures are built



Performs the case-level stratification split using an existing saved split from the command line or creating a new seeded split

```
if args.load_splits:
    # Load existing splits
    print(f"Loading existing splits from: {args.load_splits}")
    splits_data = load_data_splits(args.load_splits)
    train_cases_set = set(splits_data['train_cases'])
    val_cases_set = set(splits_data['val_cases'])
    test_cases_set = set(splits_data['test_cases'])

    # Map loaded case IDs back to slices
    train_slices = [(case_id, slice_id) for (case_id, slice_id) in slice_to_class.keys() if case_id in train_cases_set]
    val_slices = [(case_id, slice_id) for (case_id, slice_id) in slice_to_class.keys() if case_id in val_cases_set]
    test_slices = [(case_id, slice_id) for (case_id, slice_id) in slice_to_class.keys() if case_id in test_cases_set]
    print(f"Loaded splits - Train: {len(train_slices)}, Val: {len(val_slices)}, Test: {len(test_slices)}")
else:
    # Split data by case (stratified)
    train_slices, val_slices, test_slices = split_by_case_stratified(
        slices_by_class, random_state=args.seed
    )
```

Rebuilding the data into the structure the MIL model expects

Slices in split



Regroup by case

Case = MIL bag

Build MIL Case Dictionaries (final structure)

- Slices are regrouped back into cases
- All slices and patches from a case stay together
- Temporary slice labels are discarded

```
# Build case dictionaries
train_case_dict, train_label_map = build_case_dict(train_slices, patches, slice_to_class)
val_case_dict, val_label_map = build_case_dict(val_slices, patches, slice_to_class)
test_case_dict, test_label_map = build_case_dict(test_slices, patches, slice_to_class)
```

Log Annotation Clarification

Questionable Annotation in Patch Statistics

Log seemed incorrectly labelled as “mean/median patches per stain per split.” There are 79,476 patches used!

```
35
36 Mean patches per stain per split:
37 |   |   |   |
38 |   | h&e_patches | melan_patches | sox10_patches |
39 |---|---|---|
40 | split |
41 | test | 409.2 | 250.9 | 238.1 |
42 | train | 470.5 | 263.2 | 219.9 |
43 | val | 504.8 | 241.9 | 172.8 |
44
45 Values seem way too small!
46
47 Median patches per stain per split:
48 |   |   |   |
49 |   | h&e_patches | melan_patches | sox10_patches |
50 |---|---|---|
51 | split |
52 | test | 364.5 | 173.5 | 160.0 |
53 | train | 324.0 | 194.0 | 182.0 |
54 | val | 273.0 | 175.0 | 162.0 |
```

Recommendation

These values actually reflect **mean/median patches per stain per case, within each split**. So, should simply fix the annotation in the code for understanding, without changing the computation.

Traced to find the computation in the code

Found this section of the results to be traced to **print_data_summary() in utils.py**:

`groupby("split")[stain_patch_cols].mean()`
*stain_patch_cols defined to be the 3 separate stain cols
*each df row is a case
*average patch count **per case**, grouped by split (test/train/val)

```
def print_data_summary(train_df: pd.DataFrame, val_df: pd.DataFrame, test_df: pd.DataFrame):
    """Print comprehensive data summary"""
    all_df = pd.concat([train_df, val_df, test_df], ignore_index=True)

    print("\nData Summary:")
    print("=" * 50)

    # Label distribution per split
    print("\nLabel distribution per split:")
    label_dist = all_df.groupby(["split", "label"])["case_id"].nunique().unstack(fill_value=0)
    print(label_dist)

    # Patch statistics per stain per split
    stain_patch_cols = ["h&e_patches", "melan_patches", "sox10_patches"]
    print("\nMean patches per stain per split:")
    print(all_df.groupby("split")[stain_patch_cols].mean().round(1))

    print("\nMedian patches per stain per split:")
    print(all_df.groupby("split")[stain_patch_cols].median().round(1))

    # Missing stain analysis
    missing_cols = ["h&e_missing", "melan_missing", "sox10_missing"]
    print("\nMissing stain proportion:")
    print(all_df.groupby("split")[missing_cols].mean().round(3))

    # Total statistics
    print(f"\nTotal cases: {len(all_df)}")
    print(f"Total patches: {all_df['total_patches'].sum():,}")

    print("=" * 50)
```

Code Redundancies

- Everything looks good - no obvious inefficiencies or redundancies to us yet

Running the model

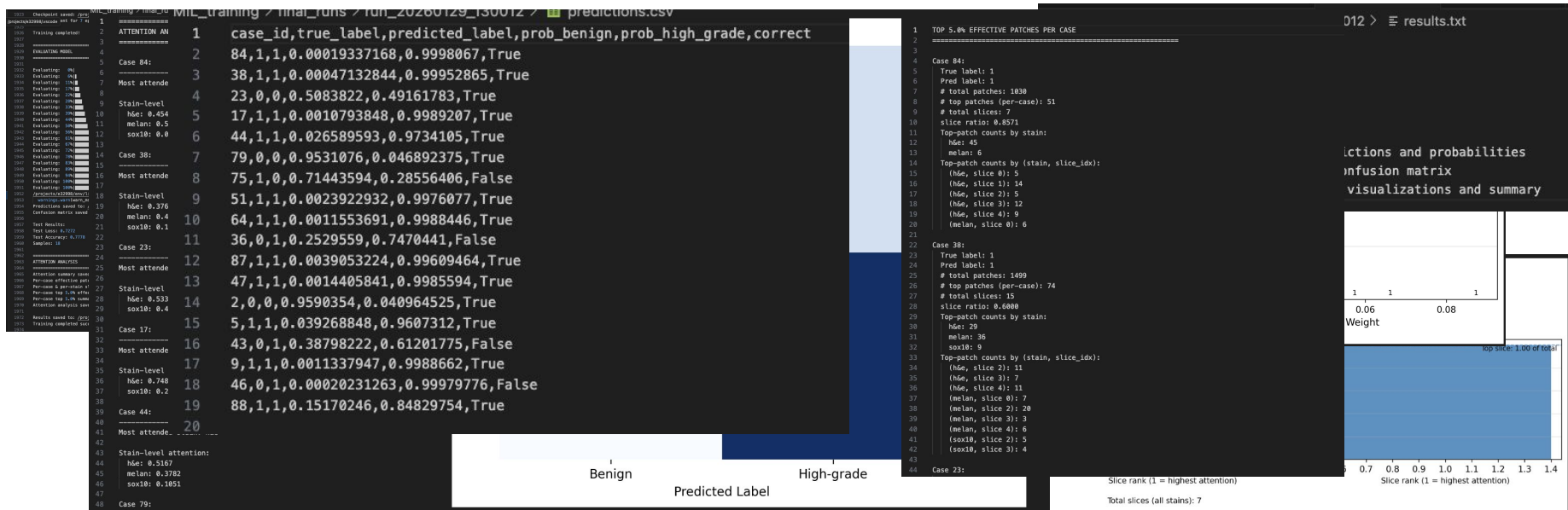
```
[rsh8958@quser42 e32998]$ cd STAT390_Winter2026/  
[rsh8958@quser42 STAT390_Winter2026]$ cd Team2  
[rsh8958@quser42 Team2]$ cd Code/MIL_trainer/sbatch_files/  
[rsh8958@quser42 sbatch_files]$ sbatch run_training_1st_splits.sbatch.save  
Submitted batch job 7273855
```

- To run the model, call the sbatch file which references main.py and has the specified command-line arguments, including the saved split to use
- Receive an email from “slurm” when job begins (and ends)
- Wait for training

```
python3 main.py \  
  --analyze_attention \  
  --attention_top_n 8 \  
  --per_slice_cap 500 \  
  --max_slices_per_stain 5 \  
  --load_splits /projects/e32998/STAT390_Winter2026/Team2/Code/MIL_trainer/data_splits/data_splits_01.npz
```

Output

Results are in the form of the training log (error and output log) and the attention_analysis (case_effective_patches, patch_attention, plots, slice_attention), confusion_matrix.png, results.txt, predictions.csv, and top_effective_patches_per_case_summary_5.0pct.txt



Part 2: Biological Background/Analysis

Adithi & Jason

2.1 Why We Need to Classify C-MIL

Clinical Problem

- Pathologists show inter-observer variability when grading conjunctival melanocytic intraepithelial lesions (C-MIL)
 - This subjectivity motivates ML support tools, but first it is necessary to clarify the biological stakes

Clinical Need

- Misdiagnosis and under-referral are common
- No universally standardized treatment pathway exists
- Better classification supports:
 - Earlier treatment, precision therapy, improved outcomes

2.2 C-MIL Progression to Conjunctival Melanoma

C-MIL as a Major Precursor

- C-MIL is the major precursor for Conjunctival Melanoma (Co-M)
 - ~70% of conjunctival melanomas arise from C-MIL
- Early detection enables:
 - Earlier intervention, risk-stratified management, improved prognosis, targeted therapy enrollment

Co-M Severity

- Rare but aggressive ocular cancer
- Treatment can lead to:
 - Vision loss, facial disfigurement, metastases and death

2.3 Co-M Epidemiology

Global Relevance

- Global incidence: ~0.46 per million per year
 - 130 new cases per year in USA, 320 in Europe
- Incidence rising (rate ratio ≈ 1.4)
- Risk increases sharply after age 65 (exponential relationship)

Co-M Outcomes

- 5-year mortality $\approx 27\%$
- Postoperative complication rates: 33–45%
- Tumor recurrence is common

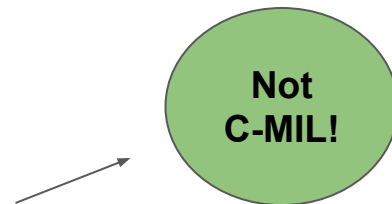
2.4 Variations in Conjunctival Melanocytic Lesion Pigment

Wide pigment variation:

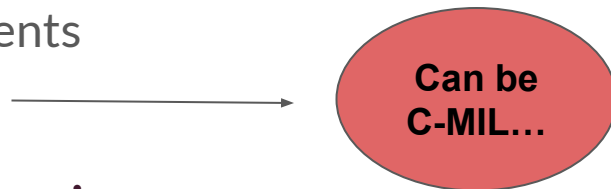
- Range from amelanotic to dark brown/black
- Pigment pattern depends on lesion type and melanocyte activity

Some pigmented conjunctival lesions are population-associated:

- Complexion-associated melanosis (CAM):
 - More common in darker-pigmented individuals
 - Typically bilateral or perilimbal with *no malignant risk*



- Primary acquired melanosis (PAM):
 - More common in middle-aged White patients
 - Often unilateral with *malignant potential*

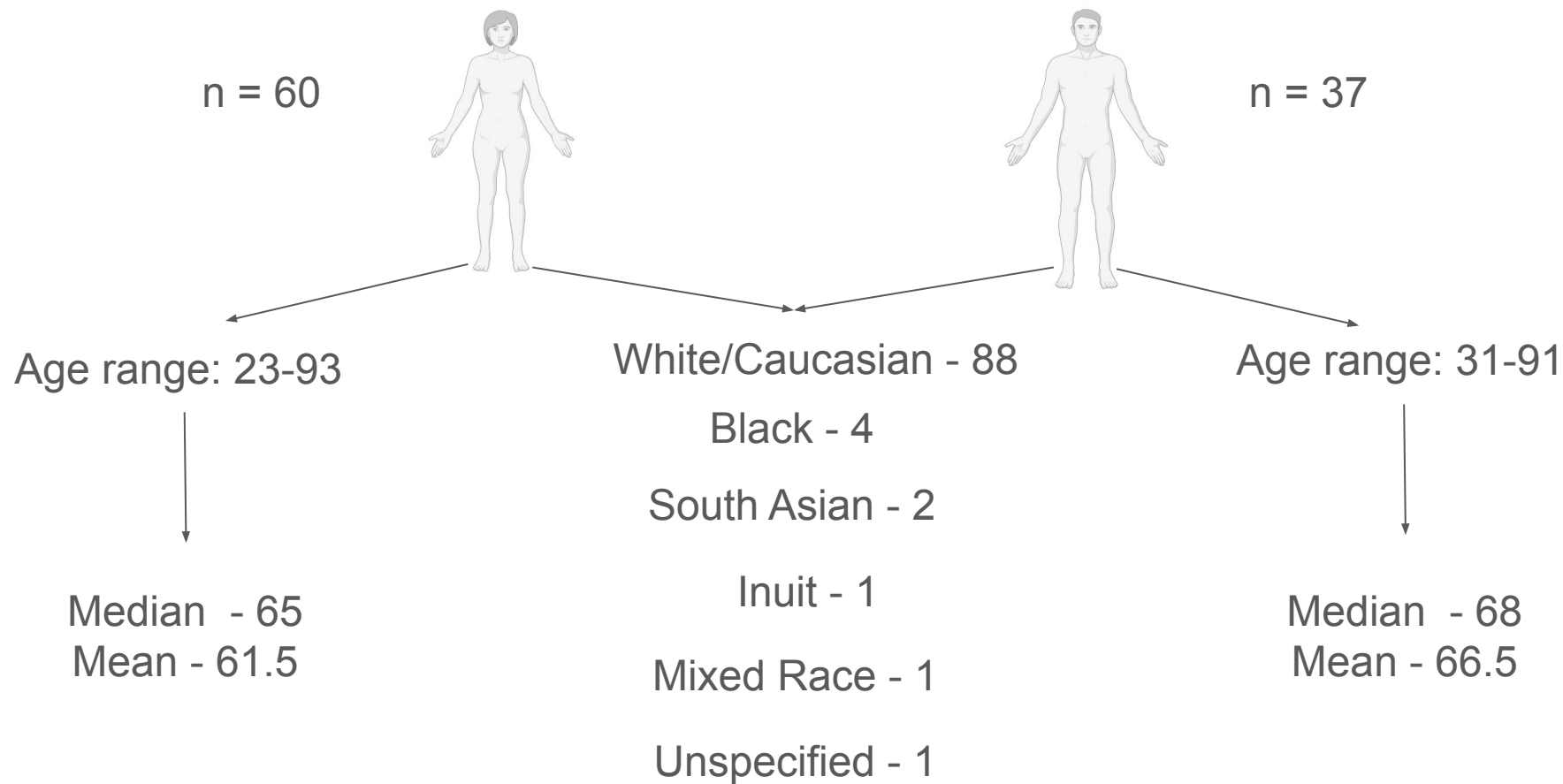


- CAM and PAM can coexist, complicating diagnosis

2.5 Overarching Risk factors for C-MIL

- 91.2% of cases in fair-skinned people
- Age
- No gender predilection
- Previous medical history
 - May be associated with Addison's disease
- UV-ray exposure
 - Consistent with UV protection by higher melanin levels
 - Connected to social background

2.6 Distribution of Dataset + Implications for C-MIL



2.7 Limitations and Potential Next Steps

Limitations

- Limited sample size
- Need to know previous medical history and age of each patient/case
- Complexity of the disease progression/human health

Potential Next Steps

- Stratifying dataset for model training with these risk factors in mind could improve accuracy
 - Allow better generalizations because the model could be getting trained by only patients within certain group
 - **Caveat:** Small sample size of patients across different demographics means sampling bias, unrepresentative, could increase variance and reduce precision

Part 3: Consultants

Model Setup

Model

- DuoHistNet
 - Efficient dual-modality transformer based framework
 - Integrates H&E and IHC-stained images for biomarker prediction

Image Acquisition:

- 40x whole-slide images (WSI) from Philips or Leica scanners
 - CRC MMR cohort: 20,820 cases
 - CRC MSI cohort: 20,879 cases
 - BRCA PD-L1 cohort: 15,173 cases
- Includes primary and metastatic specimens

Model Pipeline:

- 1. Data Preprocessing
- 2. Feature Extraction using Transformer Model
- 3. Aggregation of features to produce the final WSI-level prediction

Data Preprocessing

Tissue Segmentation:

- QuPath pixel classifiers for H&E and IHC WSIs
- YOLO-based detection to remove control tissue (TMA cores)
 - Focusing on relevant tumor regions

Tiling:

- WSIs split into 224×224 patches at 10× magnification
- Resolution: ~1 μm /pixel

Scanner-Specific Note:

- Philips WSIs: iSyntax → TIFF conversion prior to processing

Feature Extraction

Model:

- CTransPath: hybrid CNN + Swin Transformer architecture

Architecture:

- 3 initial convolution layers
 - capture local texture features and stabilize training
- 4 stages of Swin Transformer layers
 - model global contextual relationships via self-attention

Pretraining

- Self-supervised pretraining on large-scale histopathology data
 - The Cancer Genome Atlas (TCGA) and Pathology AI Platform
 - Allows model to learn generic tissue representations before fine-tuning

Output Representation

- 768-dimensional feature vector per tile → utilized for final WSI-level predictions

Aggregation & Prediction

Aggregation Model:

- Transformer-based aggregation with multi-headed self-attention
- Processes sequence of tile embeddings from a WSI
- Learns which tiles are more informative for biomarker predictions

Architecture:

- Linear projection layer → 2 transformer layers → MLP head
- Dual-branch input for H&E + IHC, with shared parameters
- Outputs from both branches concatenated and fed to classifier
- Attention scores highlight tiles contributing most to prediction

Efficiency

- Parameter sharing reduces memory usage → GPU-friendly
- Tile embeddings allow processing thousands of tiles efficiently

Model Evaluation

Performance Metrics:

- **Primary metric: AUROC**
- MMRd (CRC) predictions:
 - H&E: 0.922 | IHC: 0.947 | H&E + IHC: 0.967
- Dual-modality improves MSI and MMRd predictions

Reproducibility & Statistics:

- Nested 5-fold cross-validation (60/20/20 per fold)
- Performance averaged across folds
- Consistent model architecture, hyperparameters, and training procedures
- Retrospective real-world data → no randomization/blinding (potential bias)

Robustness:

- Minimal impact of scanner types (Philips TIFF vs Leica)
- Specimen site slightly affects performance (primary vs metastatic)

Explainability & Clinical Relevance:

- Attention heatmaps highlight predictive tissue regions
- Supports pathologist interpretation
- Allows threshold adjustment for clinical sensitivity vs specificity

Critiques/Changes

Critique	Explanation
1. Lack of Randomization & Blinding	Data is from clinical records, which affects randomization and blinding. Even if the model is “blind,” input data may reflect biased human decisions.
2. Partial Labeling (MLH1 only)	MMRd usually defined by MLH1, PMS2, MSH2, MSH6; training only on MLH1 may cause the model to replicate labels rather than discover new biological patterns.
3. Generalizability Across Scanners & Formats	Philips (iSyntax → TIFF) and Leica scanners may introduce scanner-specific artifacts; TIFF conversion may alter subtle patterns, potentially reducing performance on unseen scanners or formats.
4. Limited Tile Sampling	Using max 500 tiles per WSI per epoch may miss rare or spatially small tumor features, reducing sensitivity for certain cases.
5. Small Number of Training Epochs	10 epochs may be borderline for complex transformer models; may underfit rare subgroups or reduce reproducibility.

Possible improvements

Critique	Suggested Improvement
Lack of Randomization & Blinding	Prospective studies, blinded outcome assessment, confounder adjustment
Partial Labeling	Include full 4-marker MMR panel, multi-IHC integration
Generalizability	Multi-scanner training, domain adaptation, external validation
Limited Tile Sampling	Adaptive/stratified sampling, increase coverage, progressive sampling
Small Number of Epochs	More epochs with early stopping, gradient accumulation, mixed-precision training

References

GenAI tools

Butt, K.; Hussain, R.; Coupland, S.E.; Krishna, Y. Conjunctival Melanoma: A Clinical Review and Update. *Cancers* 2024, 16, 3121. <https://doi.org/10.3390/cancers16183121>

[https://www.pathologyoutlines.com/topic/eyepamconj.html#:~:text=Primary%20acquired%20melanosis%20\(PAM\)%20is%20also%20known,is%20significant%20factor%20for%20recurrence%20and%20progression](https://www.pathologyoutlines.com/topic/eyepamconj.html#:~:text=Primary%20acquired%20melanosis%20(PAM)%20is%20also%20known,is%20significant%20factor%20for%20recurrence%20and%20progression)

<http://aao.org/eyenet/article/conjunctival-pigmented-lesions-diagnosis-managemen#:~:text=Complexion-Associated%20Melanosis-Presentation.,pigmentation%20appears%20flat%20and%20noncystic>.

Nita Mulliqi, Kimmo Kartasalo, Henrik Olsson, Xiaoyi Ji, Lars Egevad, Martin Eklund, Pekka Ruusuvuori, OpenPhi: an interface to access Philips iSyntax whole slide images for computational pathology, *Bioinformatics*, Volume 37, Issue 21, November 2021, Pages 3995–3997, <https://doi.org/10.1093/bioinformatics/btab578>

Cheng, Y., Lama, N., Chen, M. *et al.* Synergistic H&E and IHC image analysis by AI predicts cancer biomarkers and survival outcomes in colorectal and breast cancer. *Commun Med* 5, 328 (2025). <https://doi.org/10.1038/s43856-025-01045-9>